DragonSR: equation discovery by DAG search

Symbolic regression as directed-acyclic-graph optimisation — INRIA internship, first-author paper

2026-07-31

Status Done — paper being prepared for arXiv

Role First author; method design, implementation, benchmark pipeline, paper

Team With Julie Keisler (DRAGON's author) and Anastase Charantonis, INRIA

Duration 7 months, Jan.–Jul. 2026

Context Research internship, INRIA Paris

Stack Python, PyTorch, DRAGON, XGBoost, scikit-learn (GPR), PySR

Symbolic regression searches for a closed-form equation that explains data, but tree-based genetic programming duplicates sub-expressions and degrades quickly under noise. DragonSR reformulates the search as the optimisation of a directed acyclic graph with multi-channel outputs; on 27 benchmark targets it matches or beats PySR on most noisy settings and on remote-sensing indices with 10+ input variables, where PySR recovers nothing above 0% noise.

Context

Symbolic regression (SR) sits between machine learning and scientific discovery: instead of fitting the parameters of a fixed model, it searches for both the *structure* of an equation and its constants. Most modern methods (gplearn, PySR, Operon) are genetic programming over expression trees; newer ones use reinforcement learning or LLM priors.

The starting point was DRAGON (Keisler et al., JMLR 2024), a neural architecture search framework that optimises the topology, operators and hyperparameters of a DAG with a bandit-driven evolutionary algorithm (Mutant-UCB). The research question of the internship: *can a NAS framework be repurposed to search over symbolic expressions instead of neural networks?* The applied motivation was Earth observation at INRIA: recovering spectral indices (NDVI, SAVI, …) from raw Sentinel-2 bands, i.e. noisy, multi-feature datasets that are unrepresentative of the small, clean SR benchmarks in the literature.

The technical problem

Given $\mathcal{D} = \{(x_i, y_i)\}$, find $f^\star = \arg\min_{f \in \mathcal{F}_{\mathcal{O},\mathcal{T}}} L(f, \mathcal{D})$ over the space of expressions built from operators $\mathcal{O}$ and terminals $\mathcal{T}$. Three things make this hard:

- **The search space is discrete, combinatorial and unbounded in size**, and every structure carries its own continuous constants to fit. Tree representations make it worse: $2x(x + x)$ forces a tree to recompute $x + x$ at every occurrence.
- **Noise.** Real measurements are noisy, and with thousands of samples the search must separate the analytical relationship from a heterogeneous cloud of points. PySR's recovery drops to 0/10 on every target as soon as 1% relative noise is added.
- **Dimensionality.** Remote-sensing targets have 13+ input bands; exhaustive or evolutionary search over all of them does not scale without feature selection.

Approach & architecture

Expressions are DAGs whose nodes are `(combiner, primitive operator, parameters)`; edges carry *multi-channel* tensors, so several intermediate symbolic features propagate in parallel and a sub-expression is stored once and reused. The full pipeline, on a Sentinel-2 example:

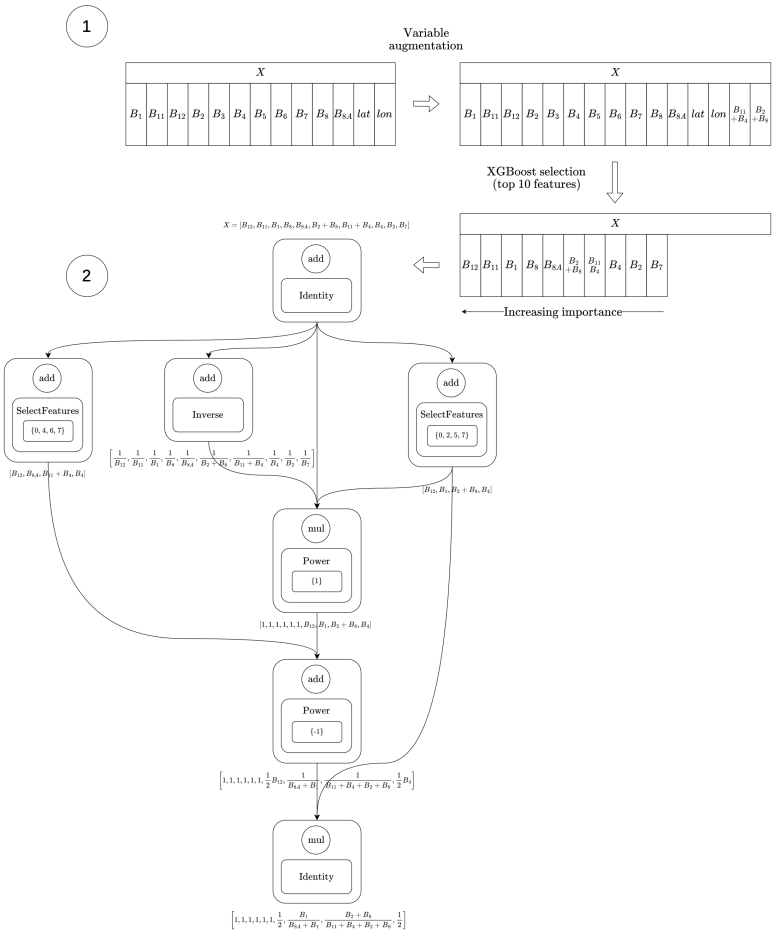


Figure 1: End-to-end DragonSR pipeline on Sentinel-2 bands: variable augmentation, XGBoost top-k selection, then evaluation of a candidate DAG whose edges are annotated with the channel representations (Figure 8 of the paper).

Main components:

1. **Preprocessing** — optional Gaussian-process denoising of the target (Matérn kernel); *variable augmentation* (apply $\{+, -, \times, \div, \sqrt{\cdot}, \log, |\cdot|, (\cdot)^2, (\cdot)^3, (\cdot)^{-1}\}$ to raw features, keep the best by R^2); XGBoost feature-importance ranking to retain the top-n features.
2. **Search space** — unary/binary primitives (powers $x^k, k \in \{\pm1, \pm2, \pm3\}$, ln, exp, sin, cos, sqrt, ...), `add`/`mul` combiners with neutral-element padding, plus two *channel* operators: `SelectFeatures` (samples a channel subset with probability $\propto$ XGBoost importance) and `ChannelBoost` (adds a learned interaction channel).
3. **Search** — DRAGON's Mutant-UCB over increasing complexity levels, 10 000 evaluations, optional parallel streams each with its own operator subset (best loss wins).
4. **Objective** — MSE, MAE, Huber or $1 - \rho_{\text{Pearson}}$ (affine-invariant, followed by an affine realignment), plus a composition penalty $\sum_{d_i \geq 2}(b^{d_i - 1} - 1)$ against nested $\sin(\sin(x))$-type chains; stochastic 10% subsampling per evaluation.
5. **Channel aggregation** — instead of taking the best single channel, combine channels with (a) sparse OLS with iterative thresholding, (b) *nested* OLS through a link function $f^{-1}(\sum_j w_j u_j(ch_j) + b)$, or (c) a polynomial-rational model linearised as $y(1 + \sum b_m \phi_m) \approx a_0 + \sum a_m \phi_m$ — all solved by linear least squares, no non-linear optimisation.

Decision	Choice made	Alternative rejected	Why
Expression representation	DAG with shared sub-expressions and variable channel width	Expression tree (GP, DSR)	No duplicated sub-expressions; channel dimension becomes a search variable, so the space is richer at no extra budget
Search algorithm	Deterministic DAG optimisation with Mutant-UCB (inherited from DRAGON)	Evolutionary CGP, exhaustive DAG enumeration (Kahlmeyer et al.)	Reuses a mature NAS optimiser; complexity-level schedule bounds expression size

Decision	Choice made	Alternative rejected	Why
Constants	Extracted post hoc by (sparse / nested / rational) least squares on the output channels	Constants inside the genome, optimised per candidate (PySR)	Turns a non-linear constant search into a linear solve per candidate; the graph only has to find the *terms*
Search loss	Pearson correlation (+ affine realignment) as default	MSE / RMSE	Affine invariance: scale and offset are free, faster convergence (Haut et al. 2022, confirmed experimentally)
Noise handling	GPR surrogate target + stochastic subsampling	MC-dropout uncertainty weighting (explored, dropped)	Simpler, fewer hyperparameters, no extra network to train
Feature selection	Variable augmentation + XGBoost importance	Search directly over all raw bands	Makes 13-band Sentinel-2 problems tractable

My contribution

- Redesigned DRAGON's search space for symbolic regression: symbolic primitives, `add`/`mul` combiners with padding, and the two channel operators `SelectFeatures` and `ChannelBoost`.
- Designed and implemented the three channel-aggregation strategies (sparse OLS, nested OLS with link functions, polynomial-rational) and the composition-depth penalty.
- Built the preprocessing chain: variable augmentation, XGBoost ranking, GPR denoising, stochastic subsampling.
- Built the unified leaderboard pipeline (dataset generation with 6 000 samples, noise injection at $\varepsilon \in \{0, 1, 5, 10\}\%$, 10 independent runs × 27 targets × 4 noise levels for both DragonSR and PySR), released on Zenodo.
- Wrote the paper as first author; explored and documented two directions that were not retained (MC-dropout weighting, IASI wet-bulb temperature retrieval).

Results

Recovery counts over 10 independent runs, expression judged equivalent to ground truth after simplification. Selected rows from Table 2 of the paper:

Target	PySR ε=0	PySR ε≥0.01	DragonSR ε=0	ε=0.01	ε=0.05	ε=0.10
Nguyen-8 ($\sqrt{x}$)	10/10	0	9/10	10/10	9/10	8/10
Nguyen-9 ($\sin x + \sin y^2$)	0/10	0	10/10	9/10	8/10	8/10
Hubble ($v = 70d$)	9/10	0	9/10	6/10	0/8	1/10
Ideal gas ($V = nRT/P$)	1/10	0	9/10	6/10	3/10	4/10
Leavitt ($M = -2.81 \log_{10} P - 1.43$)	8/10	0	10/10	6/10	7/10	5/10
Kepler 3rd	0/10	0	10/10	3/10	4/10	1/10
SAVI (Sentinel-2)	0/10	0	10/10	9/10	10/10	8/10
EVI2 (Sentinel-2)	0/10	0	9/10	8/10	3/10	3/10
BSI (Sentinel-2)	0/10	0	9/10	6/10	3/10	9/10

Target	PySR ε=0	PySR ε≥0.01	DragonSR ε=0	ε=0.01	ε=0.05	ε=0.10
AWEI_sh (Sentinel-2)	8/10	0	10/10	3/10	1/10	0/10

Takeaways: PySR is strong on simple noiseless targets (N8, N10, N11 at 10/10) but recovers **nothing** at any noise level on any of the 27 targets; DragonSR degrades gracefully up to 10% noise and is the only method recovering the multi-variable remote-sensing indices. Both fail on targets needing precise constants or deep transcendental structure (Newton, Planck, Schechter, WI2015, BAI, VARI, NIRv).

Delivered: the paper (preprint), the open DragonSR code on GitHub, and the full run-level outputs on Zenodo.

Links

- Paper: PDF — *DragonSR: Directed Acyclic Graph Search for Equation Discovery*, Chikhaoui, Keisler, Charantonis, 2026 (arXiv to come)
- Code: https://github.com/how2222/dragonsr
- Run-level data: https://doi.org/10.5281/zenodo.22146752